

## 1.Pop-Up triggered:

1. When the user clicks on a payment plan option, the click event is captured by the script's event listener attached to the document.
2. If the user clicks on the text, icon, or image inside the button, the closest() method finds the parent pricing button that contains the data-testid.
3. The script reads the data-testid of the selected pricing button using `getAttribute('data-testid')`.
4. The script checks the UPGRADE\_MAP configuration object using `if (UPGRADE_MAP[testId])` to determine whether the selected plan has an upgrade path.
5. If the data-testid exists in UPGRADE\_MAP, it means the selected plan is eligible for an upsell.
6. The script stops the original click event using `preventDefault()`, `stopPropagation()`, and `stopImmediatePropagation()`, then calls `buildPopUp(testId)` to display the upsell pop-up.

## 2.Plan selection is handled:

1. When the user clicks a pricing plan option, the click is intercepted before the site's own selection logic runs, and the upsell popup is shown instead of the plan being selected immediately.
2. The user then chooses one of two options in the popup:
  - a.Upgrade MyPlan
  - b.No Thanks
3. Based on the user's choice, `selectPlan()` is called with the appropriate plan's ID, and it locates the corresponding plan button on the page using its `data-testid` attribute.
4. Before triggering the click, the script sets `bypassIntercept = true`. This ensures the upcoming programmatic click is not caught by our own click listener again, which would otherwise reopen the popup in an endless loop.
5. The script then calls `btn.click()` on that button, simulating a genuine user click on the chosen plan.
6. Because this triggers a real click event, the website's own original selection logic runs exactly as it normally would — the plan is visually marked as selected, and the checkout flow continues normally from that point.

### **3. Assumptions Made During Development**

1. I used `data-testid` to target elements because it's more stable than CSS classes, which often change during UI updates.
2. I didn't add a `MutationObserver` because testing showed the event listener continued working even after navigation. I preferred keeping the solution simple and only adding extra logic if it became necessary.
3. To avoid an infinite loop when simulating the upgrade button click, I used a small bypass flag so the script wouldn't intercept its own click event.
4. Since the assignment focused on functionality, I kept the popup content and styling simple instead of spending time on exact UI matching.
5. I tested the script through the browser console/Snippets to verify the logic. For a real A/B test deployment, I'd also test it in the actual platform (such as Optimizely or a tag manager) to ensure the script loads at the correct time.